

AI BUILDERS HACKATHON 2026

# Veridemo Watch

Your product changed. Your published content did not. Watch finds the exact sentence that just became false.

[github.com/Ariya-rithvik/demowatch](https://github.com/Ariya-rithvik/demowatch)

## THE PROBLEM

# Published content rots silently

A creator says a plan costs one forty nine. Six weeks later it does not. The video is still up, still ranking, still converting people on a number that no longer exists — and nothing anywhere tells the creator which sentence to fix.

## THE IDEA

# A script is not prose. It is a list of claims.

```
raw script
  |
  v
segment    into individual claims
  |
  v
match      each claim to a fact
           from a live crawl
  |
  v
guard      numbers must appear
           in the source
```

Once a sentence is bound to a **fact id**, going stale is not a judgement call any more — it is a hash comparison.

---

**Drift = same fact id, different content hash.**

Re-crawl the product and Watch can name every published claim that just went false, and nothing else.

## HOW IT WORKS

# One model pass, then a check the model cannot talk its way past

### THE MODEL PASS

A single Gemini call does segmentation *and* fact-matching together, because splitting them loses the context that makes the match good.

No API key? A deterministic keyword and number-overlap matcher takes over, so the audit still produces a real result with zero setup.

### THE GUARD

Every claim the model extracts is then run through the same numeric-support check the generator uses.

---

**An LLM mismatch fails verification.** It does not ship. The model proposes; the hash decides.

## HOW IT IS BUILT

# Real infrastructure, not a prompt chain

|                            |                                                   |
|----------------------------|---------------------------------------------------|
| <code>ingest</code>        | <code>agent_ingest.py</code>                      |
| <code>orchestrator</code>  | FastAPI + priority queues                         |
| <code>job state</code>     | SQLite / Postgres                                 |
| <code>crawl</code>         | Playwright, real Chromium                         |
| <code>reasoning</code>     | Gemini, with a zero-key<br>deterministic fallback |
| <code>guard</code>         | <code>check_narration</code>                      |
| <code>agent surface</code> | MCP tool: <code>audit_content</code>              |
| <code>deploy</code>        | Zerops ( <code>zerops.yml</code> )                |

Watch is not a wrapper around a chat call. It is an agent surface: **audit\_content** is exposed over MCP, so another agent can hand it a script and get back a per-claim verdict with citations.

---

Run it end to end with no API key and no account:

```
python examples/watch_demo.py
```

A REAL RUN, CAPTURED LIVE

# The audit, as it actually runs

```
python examples/watch_demo.py
```

```
Loaded 60 facts from a real crawl of netflix.com
```

```
Creator's script:
```

```
Netflix starts at just 149 rupees a month, which is a steal for the content library. You can cancel at any time with no penalty. Honestly Netflix now costs 499 rupees for the basic plan which is way too much.
```

```
Engine: deterministic
```

```
2/3 claims verified
```

```
[PASS] Netflix starts at just 149 rupees a month, which is a steal for the content library.
```

```
[PASS] You can cancel at any time with no penalty.
```

```
[FAIL] Honestly Netflix now costs 499 rupees for the basic plan which is way too much.
```

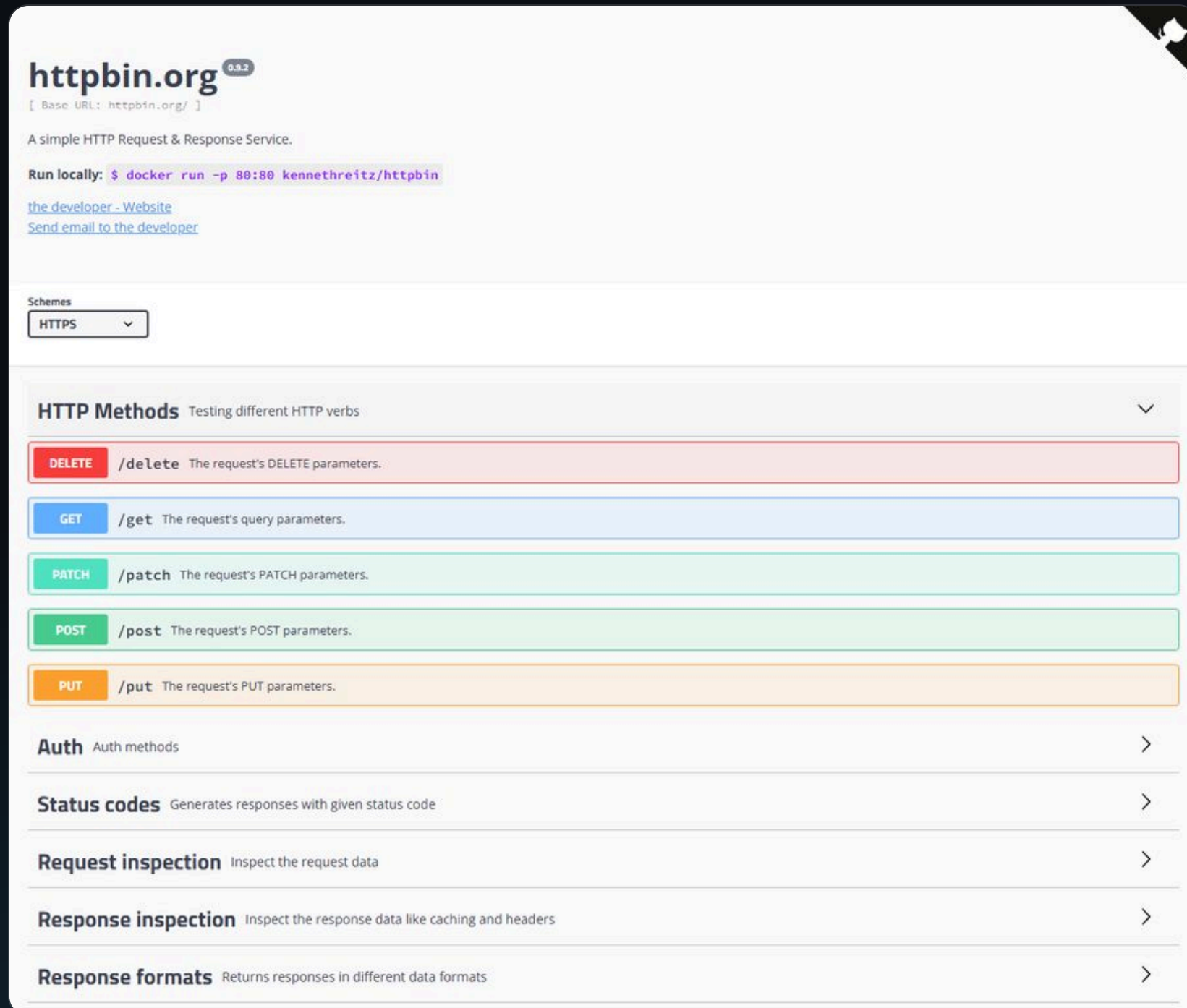
```
→ numbers not found in source: 499
```

```
OK: the outdated price claim was caught before it could ship unnoticed.
```

output captured by tools/build\_submission\_video.py – not retyped

WHERE THE FACTS COME FROM

# A crawl, not a corpus



60 facts in the Netflix truth set  
356 DOM elements hashed per crawl  
8 pages walked by a real browser

docs/proof-run/  
netflix\_truth\_set.json  
verification.json  
screenshots/

Committed to the repo. Clone it and every number in this video is reproducible.

WHO THIS IS FOR

# The back catalogue nobody re-reads

## TODAY

A creator's old videos, docs and posts keep making claims on their behalf forever. Nobody re-watches a two-year-old review to check whether its prices still hold, so the correction only ever comes from an annoyed comment.

## WITH WATCH

Every claim is bound to a fact id at publish time. Re-crawl on a schedule and the hash diff names **the exact sentence** that went false, in which asset — and the Release agent can regenerate just that wording.

Correction stops being archaeology and becomes **a diff**.

DISCLOSED, NOT HIDDEN

## Two entries, one engine

We are also submitting **Veridemo**, which uses the same verified-fact engine to *generate* demo videos. Same mechanism, different input and different user — and we would rather say so plainly than present them as unrelated.

GENERATE

[github.com/Ariya-rithvik/demo](https://github.com/Ariya-rithvik/demo)

MAINTAIN — THIS ONE

[github.com/Ariya-rithvik/demowatch](https://github.com/Ariya-rithvik/demowatch)

VERIDEMO WATCH

# Someone should be checking

Every creator has a back catalogue quietly making claims on their behalf.

Watch is the thing that **re-reads all of it every time the product ships** — and tells you the one sentence that needs to change.

```
git clone github.com/Ariya-rithvik/demowatch  
python examples/watch_demo.py
```